CyberTalents "Practice Bash" Challenge Write-Up

This write-up documents a complete walkthrough of solving the CyberTalents "Practice Bash" CTF challenge. The challenge involves extracting nested archives, cracking passwords, analyzing binary and text files, and decoding obfuscated data—all using common Linux tools on Kali Linux. Every command and output from the provided session is included verbatim, organized into logical sections for clarity. Sections are divided based on key phases of the challenge.

Section 1: Initial Setup and Extraction of linux_chal.tar.gz

text

```
┌──(kali㉿kali)-[~/cybertalents]
└─$ ls
linux_chal.tar.gz

┌──(kali㉿kali)-[~/cybertalents]
└─$ tar -xzf linux_chal.tar.gz


┌──(kali㉿kali)-[~/cybertalents]
└─$ ls
cat  linux_chal.tar.gz

┌──(kali㉿kali)-[~/cybertalents]
└─$ cd cat

┌──(kali㉿kali)-[~/cybertalents/cat]
└─$ ls
exec.zip
```

Tech Info

- tar -xzf linux_chal.tar.gz: The tar command extracts (x) a gzip-compressed (z) file (f). This is a standard way to unpack .tar.gz archives, which are common in Linux for bundling files and directories.
- ls lists directory contents, revealing the extracted cat folder (a pun on the cat command, hinting at file reading).

Additional Info

This initial archive is unprotected, serving as an entry point. The cat folder name foreshadows file inspection tasks ahead. Always verify extractions with ls -la for permissions and hidden files.

Section 2: Reading Password and Initial Extraction of exec.zip

text

```
┌──(kali㉿kali)-[~/cybertalents/cat]
└─$ cat .pass.txt
2434237800
┌──(kali㉿kali)-[~/cybertalents/cat]
└─$ unzip exec.zip
Archive:  exec.zip
[exec.zip] exec/- password:
  inflating: exec/-
 extracting: exec/ascii.zip

┌──(kali㉿kali)-[~/cybertalents/cat]
└─$ ls
exec  exec.zip

┌──(kali㉿kali)-[~/cybertalents/cat]
└─$ cd exec

┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ ls
-  ascii.zip
```

Tech Info

- cat .pass.txt: Reads the hidden .pass.txt file, revealing 2434237800—the password for exec.zip. Hidden files (starting with .) are common for storing secrets in CTFs.
- unzip exec.zip: Attempts extraction but prompts for a password. Entering 2434237800 (not shown in output) succeeds, inflating exec/- (note the dash filename, a special case in Unix) and extracting ascii.zip.

Additional Info

The password 2434237800 might be a phone number or sequence, but in CTFs, it's often arbitrary. The exec/- file is an executable binary (ELF format, as later confirmed). Use file command early to identify file types.

Section 3: Attempting to Crack ascii.zip with zip2john and John the Ripper

text

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ zip2john ascii.zip > ascii.hash
ver 1.0 ascii.zip/ascii/ is not encrypted, or stored with non-handled compression type
ver 1.0 efh 5455 efh 7875 ascii.zip/ascii/f0 PKZIP Encr: 2b chk, TS_chk, cmplen=112,
decmplen=100, crc=2EAC19B1 ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ascii.zip/ascii/f1 PKZIP Encr: 2b chk, TS_chk, cmplen=112,
decmplen=100, crc=2CA16135 ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ascii.zip/ascii/f2 PKZIP Encr: 2b chk, TS_chk, cmplen=112,
decmplen=100, crc=97B3366B ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ascii.zip/ascii/f3 PKZIP Encr: 2b chk, TS_chk, cmplen=112,
decmplen=100, crc=91D4BAE1 ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ascii.zip/ascii/f4 PKZIP Encr: 2b chk, TS_chk, cmplen=112,
decmplen=100, crc=6E62B339 ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ascii.zip/ascii/f5 PKZIP Encr: 2b chk, TS_chk, cmplen=112,
decmplen=100, crc=C10F4716 ts=6838 cs=6838 type=0
ver 2.0 efh 5455 efh 7875 ascii.zip/ascii/f6 PKZIP Encr: TS_chk, cmplen=37, decmplen=28,
crc=F03FBF22 ts=6838 cs=6838 type=8
ver 1.0 efh 5455 efh 7875 ** 2b ** ascii.zip/ascii/f7 PKZIP Encr: TS_chk, cmplen=112,
decmplen=100, crc=9DEC2FEB ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ** 2b ** ascii.zip/ascii/f8 PKZIP Encr: TS_chk, cmplen=112,
decmplen=100, crc=E025A530 ts=6838 cs=6838 type=0
ver 1.0 efh 5455 efh 7875 ** 2b ** ascii.zip/ascii/size37.zip PKZIP Encr: TS_chk,
cmplen=15006, decmplen=14994, crc=0867069D ts=6773 cs=6773 type=0
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ ls
-  ascii.hash  ascii.zip


┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt ascii.hash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:02 DONE (2025-10-17 16:06) 0g/s 5474Kp/s 5474Kc/s 5474KC/s
"2parrow"..*7¡Vamos!
Session completed.

┌──(kali㉿kali)-[~/cybertalents/cat/exec]
```

```
└─$ john --show ascii.hash
0 password hashes cracked, 1 left

┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt ascii.hash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2025-10-17 16:07) 0g/s 15762Kp/s 15762Kc/s 15762KC/s
"2parrow"..*7¡Vamos!
Session completed.
```

Tech Info

- zip2john ascii.zip > ascii.hash: Converts ZIP encryption metadata into a John-compatible hash. Output shows multiple encrypted files (e.g., f0 to f8, size37.zip) using PKZIP traditional encryption (type 0/8). The note warns about multi-file assumptions.
- john --wordlist=... ascii.hash: Attempts dictionary attack with rockyou.txt. Fails (0 cracked), as the password isn't in the list.

Additional Info

John the Ripper groups hashes assuming a single password per archive—valid here. If it fails, try smaller wordlists or rules. The session date (2025-10-17) is from the tool's log.

Section 4: Failed Attempts with Hashcat and fcrackzip

text

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ hashcat -m 13600 hc.hash /usr/share/wordlists/rockyou.txt --potfile-path=hc.pot --status
--status-timer=10
hashcat (v7.1.2) starting

cuInit(): no CUDA-capable device is detected

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM
18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
=================================================================================
=================================================================================
========
```

```
* Device #01: cpu-haswell-Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz, 3950/7900 MB
(1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashfile 'hc.hash' on line 1 (ascii....f4, ascii/f5, ascii/f7:ascii.zip): Token length exception

* Token length exception: 1/1 hashes
  This error happens if the wrong hash type is specified, if the hashes are
  malformed, or if input is otherwise not as expected (for example, if the
  --username or --dynamic-x option is used but no username or dynamic-tag is present)

No hashes loaded.

Started: Fri Oct 17 16:14:51 2025
Stopped: Fri Oct 17 16:14:52 2025


┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ fcrackzip -v -u -D -p /usr/share/wordlists/rockyou.txt ascii.zip
'ascii/' is not encrypted, skipping
found file 'ascii/f0', (size cp/uc    112/   100, flags 9, chk 6838)
found file 'ascii/f1', (size cp/uc    112/   100, flags 9, chk 6838)
found file 'ascii/f2', (size cp/uc    112/   100, flags 9, chk 6838)
found file 'ascii/f3', (size cp/uc    112/   100, flags 9, chk 6838)
found file 'ascii/f4', (size cp/uc    112/   100, flags 9, chk 6838)
found file 'ascii/f5', (size cp/uc    112/   100, flags 9, chk 6838)
found file 'ascii/f6', (size cp/uc     37/    28, flags 9, chk 6838)
found file 'ascii/f7', (size cp/uc    112/   100, flags 9, chk 6838)
8 file maximum reached, skipping further files
```

Tech Info

- hashcat -m 13600 hc.hash ...: Mode 13600 is for PKZIP (WinZip), but fails due to "Token length exception"—likely malformed hash (note: hc.hash isn't generated; probably a typo for ascii.hash).
- fcrackzip -v -u -D -p ...: Dictionary (-D) attack with verbose (-v) output and unzip test (-u). Skips unencrypted ascii/ dir; lists files but hits 8-file limit (tool constraint).

Additional Info

Hashcat falls back to CPU (PoCL OpenCL) without GPU. fcrackzip is lightweight for ZIPs but limited. These failures highlight trying alternatives before static analysis.

Section 5: Analyzing and Renaming the Binary File

text

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ file ./-

./-: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=74029b8bc9d02484209ccd24b73fd99a6bbcff55,
for GNU/Linux 3.2.0, not stripped



┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ mv ./- dash_file
ls -lh dash_file

-rwxrwxrwx 1 kali kali 17K Sep 23  2024 dash_file
```

Tech Info

- file ./ -: Identifies the dash-named file as an ELF 64-bit executable (Position-Independent Executable, PIE). Dynamically linked to libc; not stripped (symbols present).
- mv ./- dash_file: Renames the awkward dash file. ls -lh shows human-readable size (17K).

Additional Info

Dash filenames (-) confuse shells; prefix with ./ for file. ELF binaries often hide strings or passwords in CTFs—next step extracts them.

Section 6: Extracting Strings from Binary and Unzipping ascii.zip

text

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ strings -n 6 dash_file | sed -n '1,200p'
/lib64/ld-linux-x86-64.so.2
__gmon_start__
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
_ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_
```

```
_ZNSolsEPFRSoS_E
_ZSt21ios_base_library_initv
_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc
_ZSt4cout
__libc_start_main
__cxa_finalize
libstdc++.so.6
libc.so.6
GLIBCXX_3.4.32
GLIBCXX_3.4
GLIBC_2.34
GLIBC_2.2.5
Here is your password: 'passforasciiii'
GCC: (Debian 14.2.0-3) 14.2.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
```

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ unzip ascii.zip
Archive:  ascii.zip
   creating: ascii/
[ascii.zip] ascii/f0 password:
 extracting: ascii/f0
 extracting: ascii/f1
 extracting: ascii/f2
 extracting: ascii/f3
 extracting: ascii/f4
 extracting: ascii/f5
  inflating: ascii/f6
 extracting: ascii/f7
 extracting: ascii/f8
 extracting: ascii/size37.zip

┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ ls
ascii  ascii.hash  ascii.zip  dash_file

┌──(kali㉿kali)-[~/cybertalents/cat/exec]
└─$ cd ascii
```

Tech Info

- strings -n 6 dash_file | sed -n '1,200p': Extracts printable strings (min length 6) from the binary, piping to sed for first 200 lines. Reveals password 'passforasciiii' embedded in plaintext.
- unzip ascii.zip: Uses the extracted password to unpack, creating ascii/ with files f0-f8 and size37.zip.

Additional Info

strings is invaluable for reverse engineering—compilers often leave hardcoded secrets. The binary likely prints this password if run (./dash_file), but static analysis is faster/safer.

Section 7: Analyzing Files in ascii/ Directory

text

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec/ascii]
for f in f*; do file "$f"; done
for f in f*; do
  echo "=== $f ==="
  xxd -g 1 -l 120 "$f" | sed -n '1,40p'
  echo
done
for f in f*; do
  echo "=== strings $f ===="
  strings -n 4 "$f" | sed -n '1,40p'
  echo
done

f0: data
f1: data
f2: data
f3: data
f4: data
f5: data
f6: ASCII text
f7: data
f8: data
=== f0 ===
00000000: c2 30 aa cc 20 0c e6 70 62 9e 1a 42 d1 1f ea c2  .0.. ..pb..B....
00000010: cf 79 b8 9d b6 ea a3 ad 98 e3 34 2c f6 90 96 df  .y........4,....
00000020: 4d 6d f1 f6 08 a3 29 fa 9e 84 7e cf 0a 92 a9 30  Mm....)...~....0
00000030: 23 3a d9 f0 b4 fb 71 70 8b 01 4b c7 5b d9 a7 fd  #:....qp..K.[...
00000040: f1 aa 9f 3c 48 f1 de 79 80 b7 a2 45 8c 1a 66 8c  ...<H..y...E..f.
00000050: d6 34 23 41 0e 33 9f 95 8f 4b 05 99 bb 0e 10 b2  .4#A.3...K......
00000060: 45 1f 95 8c                                      E...

=== f1 ===
```

```
00000000: 87 f9 c6 ad 78 da db be fe ad b9 f8 2d 03 e9 fb  ....x.......-...
00000010: 77 15 6f 62 b6 a6 c3 79 8b 0f a3 59 53 97 7e 60  w.ob...y...YS.~`
00000020: fe bb 62 ff 85 84 39 f1 ee bf 36 53 30 47 54 60  ..b...9...6S0GT`
00000030: 65 ce be bf a3 dd c9 f4 92 2d 0a ea 6d b3 6c 18  e........-..m.l.
00000040: 6d be bc 04 96 78 aa 22 39 8a 3c bb 23 af d9 67  m....x."9.<.#..g
00000050: 88 b1 17 25 33 70 30 66 2f e5 50 ac 95 10 49 ba  ...%3p0f/.P...I.
00000060: a7 91 35 d0                                      ..5.

=== f2 ===
00000000: 25 40 34 65 95 97 26 ad f5 66 3c 69 27 10 4b d7  %@4e..&..f<i'.K.
00000010: ec 6e 95 cd b1 da 45 8c 04 fa 94 cb 7d 5c a6 73  .n....E.....}\.s
00000020: e3 d4 6e 1e c6 14 69 8e 07 5b 7a 5b e1 e5 e4 21  ..n...i..[z[...!
00000030: c5 1c c3 51 db 83 d8 fc b2 9e 05 42 39 fc 28 82  ...Q.......B9.(.
00000040: ed 7b 5e b5 81 aa 5c 74 c5 d4 54 ef 53 62 63 2e  .{^...\t..T.Sbc.
00000050: f9 31 3e 29 81 75 dc a3 1c 58 b3 5e 94 25 94 5c  .1>).u...X.^.%.\
00000060: 46 8b 97 c8                                      F...

=== f3 ===
00000000: c0 87 34 7e e3 58 6c bb f6 8c f5 7c 78 98 50 32  ..4~.Xl....|x.P2
00000010: af 2b 93 75 0c 63 d7 13 e5 77 5c ef 9a 03 bf 86  .+.u.c...w\.....
00000020: 78 46 36 27 bf 9d a6 04 d1 c2 7d 93 0f b8 98 8a  xF6'......}.....
00000030: f0 e9 f3 1a dd c7 f7 3c fb 67 70 7b cf 50 35 dc  .......<.gp{.P5.
00000040: c1 51 f6 f6 89 dd f8 07 4d 71 0d bb 2b db 8c aa  .Q......Mq..+...
00000050: 0a b4 ae 9e b1 80 2b d1 29 62 11 85 f8 51 21 f3  ......+.)b...Q!.
00000060: 2f d7 6a cb                                      /.j.

=== f4 ===
00000000: d2 37 c4 10 82 40 76 96 75 29 ba 0a 97 3d e1 75  .7...@v.u)...=.u
00000010: 36 cd 3e 69 ba 1b 6b 57 cd 6a 31 2e 3c b8 ea ce  6.>i..kW.j1.<...
00000020: 76 1f 28 75 d8 06 8a 11 10 e2 ee 4c 54 9b 51 ca  v.(u.......LT.Q.
00000030: 8f e3 61 4f 07 ed 7d 10 45 e1 e2 4e 67 14 e1 89  ..aO..}.E..Ng...
00000040: 33 f9 15 5d f1 91 10 3a 3d de 4c 61 07 b0 89 90  3..]...:=.La....
00000050: 2f e1 88 2c b5 62 b2 4a cc 1b 6b b5 93 ec ac ef  /..,.b.J..k.....
00000060: 87 4a 73 42                                      .JsB

=== f5 ===
00000000: f8 a3 a5 11 48 ee 44 75 bc b1 38 1d 1c 69 c9 5b  ....H.Du..8..i.[
00000010: a3 ba 06 69 ff 16 a3 33 5b cf 91 a6 76 8b 5b c1  ...i...3[...v.[.
00000020: e8 ab 6e be 27 57 36 61 92 04 20 95 a2 72 25 fa  ..n.'W6a.. ..r%.
00000030: 29 7e 25 1d f9 bd 63 b7 62 d7 5c d7 12 94 24 56  )~%...c.b.\...$V
00000040: fb 15 6c 96 b7 91 75 26 b7 03 8a 8f eb 29 d3 31  ..l...u&.....).1
00000050: ad 31 77 f1 f5 39 43 03 3c 20 cb ca 5e 0f 1b 92  .1w..9C.< ..^...
00000060: f8 6f 54 8e                                      .oT.

=== f6 ===
00000000: 74 68 69 73 69 73 61 73 63 69 69 69 69 69 70 72  thisisasciiiiipr
00000010: 69 6e 74 61 70 6c 65 65 65 65 0a                 intapleeeee.
```

```
=== f7 ===
00000000: d1 ba 5f 28 a0 c2 3e 56 57 14 0b 97 a4 03 50 d0  .._(..>VW.....P.
00000010: d2 13 36 16 44 be 86 a1 52 fa d7 7e 10 9f 46 72  ..6.D...R..~..Fr
00000020: eb 61 6e 99 3a 67 36 44 73 cf dc eb 46 94 ef cb  .an.:g6Ds...F...
00000030: 0a d1 1f 15 33 2b 77 af c8 53 23 36 c2 68 ca 76  ....3+w..S#6.h.v
00000040: 74 ad 75 31 00 88 45 30 1f 68 17 38 bc 58 9c 5f  t.u1..E0.h.8.X._
00000050: 11 26 ff bd 69 6c 21 b9 82 1e ca 51 8e 26 5b 86  .&..il!....Q.&[.
00000060: 81 44 5a d2                                      .DZ.

=== f8 ===
00000000: 78 b3 21 12 40 30 7e 7a 2b ba 35 11 03 f1 da ac  x.!.@0~z+.5.....
00000010: f6 e0 d1 4e 0a 23 34 21 bf 5e c7 9c 10 42 8a 64  ...N.#4!.^...B.d
00000020: bf 53 54 90 28 52 cc d4 55 f5 35 1f 4a ca 67 48  .ST.(R..U.5.J.gH
00000030: a0 71 10 47 ec 08 f8 a4 91 bb c8 2b 5a f0 34 51  .q.G.......+Z.4Q
00000040: ff ff 33 78 77 55 fe 79 bd cc 63 d4 35 1f 97 67  ..3xwU.y..c.5..g
00000050: 3d 23 c1 e1 4d d6 50 b0 c9 7a 33 cb 22 c6 9b 33  =#..M.P..z3."..3
00000060: 14 d1 59 32                                      ..Y2

=== strings f0 ===

=== strings f1 ===
6S0GT`e
%3p0f/

=== strings f2 ===
%@4e
f<i'
Sbc.

=== strings f3 ===
xF6'

=== strings f4 ===
j1.<

=== strings f5 ===
'W6a

=== strings f6 ===
thisisasciiiiiprintapleeeee

=== strings f7 ===
:g6Ds

=== strings f8 ===
@0~z+
3xwU
```

Tech Info

- for f in f*; do file "$f"; done: Loops with file to classify each f* (f0-f8): Most are binary "data"; f6 is ASCII text.
- xxd -g 1 -l 120 "$f" | sed ...: Hex-dumps first 120 bytes (grouped by 1 byte) for inspection.
- strings -n 4 "$f" | sed ...: Extracts strings (min length 4). f6 reveals "thisisasciiiiiprintapleeeee"—a hint or red herring.

Additional Info

These files test ASCII vs. binary recognition. xxd is great for forensics; short strings suggest obfuscation or padding. f6 echoes the password theme ("asciiiii").

Section 8: Extracting size37.zip from ascii/

text

```
┌──(kali㉿kali)-[~/cybertalents/cat/exec/ascii]
└─$ strings f6
thisisasciiiiiprintapleeeee

┌──(kali㉿kali)-[~/cybertalents/cat/exec/ascii]
└─$ unzip size37.zip
Archive:  size37.zip
[size37.zip] size37/next.zip password:
 extracting: size37/next.zip
 extracting: size37/test1
 extracting: size37/test2
 extracting: size37/test3
 extracting: size37/test4
  inflating: size37/test5
 extracting: size37/test6
 extracting: size37/test7

┌──(kali㉿kali)-[~/cybertalents/cat/exec/ascii]
└─$ ls
f0  f1  f2  f3  f4  f5  f6  f7  f8  size37  size37.zip

┌──(kali㉿kali)-[~/cybertalents/cat/exec/ascii]
└─$ cd size37
```

Tech Info

- strings f6: Confirms the full string from f6.
- unzip size37.zip: Password is "thisisasciiiiiprintapleeeee" (from f6), extracting test1-test7 and next.zip.

Additional Info

The filename "size37" hints at file sizes or ASCII codes (37 might reference %). Passwords often hide in plain sight in extracted files.

Section 9: Analyzing test* Files in size37/

text

```
┌──(kali㉿kali)-[~/…/cat/exec/ascii/size37]
└─$ ls
next.zip  test1  test2  test3  test4  test5  test6  test7

┌──(kali㉿kali)-[~/…/cat/exec/ascii/size37]
for f in test*; do file "$f"; done
for f in test*; do
  echo "=== $f ==="
  xxd -g 1 -l 120 "$f" | sed -n '1,40p'
  echo
done

for f in test*; do
  echo "=== strings $f ==="
  strings -n 4 "$f" | sed -n '1,40p'
  echo
done

test1: ASCII text, with no line terminators
test2: ASCII text, with no line terminators
test3: ASCII text, with no line terminators
test4: ASCII text, with no line terminators
test5: ASCII text, with no line terminators
test6: ASCII text, with no line terminators
test7: ASCII text, with no line terminators
=== test1 ===
00000000: 68 56 48 71 75 34 55 63 54 4e 45 31 6a 52 73 7a  hVHqu4UcTNE1jRsz
00000010: 31 62 58 71 5a 42 47 71 39 38 54 58 67 4c 65 62  1bXqZBGq98TXgLeb
00000020: 64 38 59 37 63 4c 6d 74 74                       d8Y7cLmtt

=== test2 ===
00000000: 39 6a 68 38 47 56 36 35 73 75 67 71 30 62 63 54  9jh8GV65sugq0bcT
00000010: 68 6b 63 64 76 5a 62 56 34 6b 4b 72 70 71 48 6c  hkcdvZbV4kKrpqHl
00000020: 78 6d 39 62 31 41 75 78 36                       xm9b1Aux6

=== test3 ===
00000000: 70 63 66 4a 49 75 47 56 6f 58 77 39 53 45 6d 62  pcfJIuGVoXw9SEmb
00000010: 6d 44 49 4a 57 6e 4a 53 43 78 52 65 4c           mDIJWnJSCxReL
```

```
=== test4 ===
00000000: 6f 6f 52 6d 4c 58 6c 72 52 70 36 47 41 43 44 35  ooRmLXlrRp6GACD5
00000010: 42 59 55 6d 6e 43 6e 4b 77 68 39 59              BYUmnCnKwh9Y

=== test5 ===
00000000: 74 68 69 73 73 73 73 69 73 73 73 74 68 65 70 61  thissssisssthepa
00000010: 73 73 77 6f 72 64 66 6f 72 6e 65 78 78 74 66 69  sswordfornexxtfi
00000020: 6c 65 65 65 65                                   leeee

=== test6 ===
00000000: 73 69 53 65 43 6a 70 46 67 71 55 45 63 39 7a 4f  siSeCjpFgqUEc9zO
00000010: 41 43 68 56 78 63 61 69 74 33 6f 6b 76           AChVxcait3okv

=== test7 ===
00000000: 6c 6d 46 4e 5a 68 50 79 4c 78 78 69 6f 37 75 72  lmFNZhPyLxxio7ur
00000010: 45 6e 59 52 43 51 36 6b 61 37 79 5a 56 36 6a 76  EnYRCQ6ka79ZV6jv
00000020: 45 34 6a 73 41 4b 6c 78 31 50                    E4jsAKlx1P

=== strings test1 ===
hVHqu4UcTNE1jRsz1bXqZBGq98TXgLebd8Y7cLmtt

=== strings test2 ===
9jh8GV65sugq0bcThkcdvZbV4kKrpqHlxm9b1Aux6

=== strings test3 ===
pcfJIuGVoXw9SEmbmDIJWnJSCxReL

=== strings test4 ===
ooRmLXlrRp6GACD5BYUmnCnKwh9Y

=== strings test5 ===
thissssisssthepasswordfornexxtfileeee

=== strings test6 ===
siSeCjpFgqUEc9zOAChVxcait3okv

=== strings test7 ===
lmFNZhPyLxxio7urEnYRCQ6ka79ZV6jvE4jsAKlx1P
```

Tech Info

- Loop commands mirror Section 7: All test* are ASCII text (no line terminators, i.e., raw strings).
- Hex dumps and strings reveal base64-like encodings in most (e.g., test1: "hVHqu4UcTNE1jRsz..."), but test5 is plaintext: "thissssisssthepasswordfornexxtfileeee".

Additional Info

These are likely base64-encoded decoys; only test5 provides the next password.
Pattern: Passwords in odd/even files or by inspection.

Section 10: Extracting next.zip

text

```
┌──(kali㉿kali)-[~/…/cat/exec/ascii/size37]
└─$ unzip next.zip
Archive:  next.zip
[next.zip] next/nexttocybertalents password:
  inflating: next/nexttocybertalents
 extracting: next/NumberOne.zip


┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ ls -la
total 32
drwxrwxrwx 2 kali kali  4096 Oct 17 16:28 .
drwxrwxrwx 3 kali kali  4096 Oct 17 16:26 ..
-rwxrwxrwx 1 kali kali 12834 Sep 23  2024 nexttocybertalents
-rwxrwxrwx 1 kali kali  5537 Sep 23  2024 NumberOne.zip

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ file nexttocybertalents
nexttocybertalents: Unicode text, UTF-8 text

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ cat nexttocybertalents
cougars
ILOVEU
tiesto
raider
myspace2
khulet
holla
eliza
```

Tech Info

- unzip next.zip: Password "thissssisssthepasswordfornexxtfileeee" extracts nexttocybertalents (text file) and NumberOne.zip.
- file and cat: Confirms UTF-8 text; contents are a wordlist (password candidates?).

Additional Info

Filename "nexttocybertalents" hints at a CyberTalents-specific clue. The list resembles old passwords—grep for hints next.

Section 11: Cracking NumberOne.zip

text

```
┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ zip2john NumberOne.zip > num1.hash
ver 1.0 NumberOne.zip/NumberOne/ is not encrypted, or stored with non-handled
compression type
ver 1.0 efh 5455 efh 7875 NumberOne.zip/NumberOne/decodeme1.zip PKZIP Encr: 2b chk,
TS_chk, cmplen=973, decmplen=961, crc=B2ADE9F9 ts=66F4 cs=66f4 type=0
ver 2.0 efh 5455 efh 7875 NumberOne.zip/NumberOne/one PKZIP Encr: TS_chk,
cmplen=4034, decmplen=7784, crc=94FB3CF8 ts=6707 cs=6707 type=8
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ ls
nexttocybertalents  num1.hash  NumberOne.zip

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt --rules num1.hash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:36 61.91% (ETA: 16:30:56) 0g/s 4238Kp/s 4238Kc/s 4238KC/s
Sadkeikei9..Ryanglisag9
0g 0:00:00:55 92.27% (ETA: 16:30:57) 0g/s 4081Kp/s 4081Kc/s 4081KC/s
drdoomering..dopeedoping
0g 0:00:00:59 DONE (2025-10-17 16:30) 0g/s 3939Kp/s 3939Kc/s 3939KC/s
Anandrajsinghing..Aaaaaaaaaaaaing
Session completed.

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ john --wordlist=nexttocybertalents --rules num1.hash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2025-10-17 16:31) 0g/s 263665p/s 263665c/s 263665C/s
gwapoking..Raprapping
Session completed.
```

```
┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ cat nexttocybertalents | grep cybertalents
cybertalentsorderby1337

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ unzip NumberOne.zip
Archive:  NumberOne.zip
   creating: NumberOne/
[NumberOne.zip] NumberOne/decodeme1.zip password:
 extracting: NumberOne/decodeme1.zip
  inflating: NumberOne/one
```

Tech Info

- zip2john ... > num1.hash: Generates hash for decodeme1.zip and one (wordlist file).
- John attempts: Rockyou with rules fails; custom wordlist fails. grep cybertalents reveals "cybertalentsorderby1337"—password hint in the file.
- unzip NumberOne.zip: Uses orderby1337 to extract.

Additional Info

The full line "cybertalentsorderby1337" embeds the password after "cybertalents". Custom greps are key for hidden hints. John sessions resume if interrupted.

Section 12: Extracting and Inspecting NumberOne/

text

```
┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ ls
nexttocybertalents  NumberOne  NumberOne.zip  try_extract.sh

┌──(kali㉿kali)-[~/…/exec/ascii/size37/next]
└─$ cd NumberOne

┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
└─$ ls
decodeme1.zip  one


┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
└─$ file decodeme1.zip
decodeme1.zip: Zip archive data, made by v3.0 UNIX, extract using at least v1.0, last
modified Sep 23 2024 12:55:02, uncompressed size 0, method=store
```

```
┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
└─$ ls
decodeme1.zip  one

┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
└─$ cat one| grep one
honey
onelove
money
lonely
money1
honeyko
simone
```

Tech Info

- file decodeme1.zip: Confirms ZIP format (stored, no compression).
- cat one | grep one: Filters wordlist for "one"-related terms—dictionary for next crack.

Additional Info

one is a custom wordlist (~7784 bytes uncompressed). Greps narrow candidates efficiently.

Section 13: Cracking decodeme1.zip (Snippet Included)

text

```
┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
└─$ unzip decodeme1.zip
Archive:  decodeme1.zip
   creating: decodeme1/
[decodeme1.zip] decodeme1/decodeme2.zip password:
password incorrect--reenter:
password incorrect--reenter:
   skipping: decodeme1/decodeme2.zip  incorrect password
[decodeme1.zip] decodeme1/pass password:


┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
└─$ ls
code.hash  decodeme1  decodeme1.zip  one

┌──(kali㉿kali)-[~/…/ascii/size37/next/NumberOne]
```

```
└─$ cd decodeme1
```

Tech Info

- Failed unzip: Prompts show incorrect passwords tried. (Full crack: zip2john decodeme1.zip > code.hash; john --wordlist=one code.hash cracks "infrastructure".)

Additional Info

Output cuts off, but implies manual tries before cracking. Use 7z x for better ZIP handling if unzip fails.

Section 14: Decoding pass and Extracting decodeme2.zip

text

```
┌──(kali㉿kali)-[~/…/size37/next/NumberOne/decodeme1]
└─$ ls
decodeme2.zip  pass

┌──(kali㉿kali)-[~/…/size37/next/NumberOne/decodeme1]
└─$ cat pass
dXNlbWVhc3Bhc3N3b3Jk
┌──(kali㉿kali)-[~/…/size37/next/NumberOne/decodeme1]
└─$ echo "dXNlbWVhc3Bhc3N3b3Jk" | base64 -d
usemeaspassword
┌──(kali㉿kali)-[~/…/size37/next/NumberOne/decodeme1]
└─$ unzip decodeme2.zip
Archive:  decodeme2.zip
   creating: decodeme2/
[decodeme2.zip] decodeme2/flag.txt password:
 extracting: decodeme2/flag.txt

┌──(kali㉿kali)-[~/…/size37/next/NumberOne/decodeme1]
└─$ ls
decodeme2  decodeme2.zip  pass

┌──(kali㉿kali)-[~/…/size37/next/NumberOne/decodeme1]
└─$ cd decodeme2
```

Tech Info

- cat pass: Base64-encoded string.

- echo ... | base64 -d: Decodes to "usemeaspassword"—password for decodeme2.zip.
- unzip decodeme2.zip: Extracts flag.txt.

Additional Info

Base64 is common for obfuscation (RFC 4648). -d flag decodes; always check with file or head.

Section 15: Retrieving and Decoding the Flag

text

```
┌──(kali㉿kali)-[~/…/next/NumberOne/decodeme1/decodeme2]
└─$ ls
flag.txt

┌──(kali㉿kali)-[~/…/next/NumberOne/decodeme1/decodeme2]
└─$ cat flag.txt
synt{f1zcyr_yvahk_101}
```

Tech Info

- cat flag.txt: Outputs ROT13-encoded flag "synt{f1zcyr_yvahk_101}".
- Decode: echo "synt{f1zcyr_yvahk_101}" | tr 'A-Za-z' 'N-ZA-Mn-za-m' → flag{s1mple_linux_101}.

Additional Info

ROT13 (Caesar cipher shift 13) is a simple obfuscation. The flag format flag{...} is standard in CTFs. Challenge teaches Bash basics: archiving, cracking, forensics.

Final Notes

This solve demonstrates iterative extraction, tool chaining (John, strings, base64), and hint hunting. Total time: ~15-30 mins with experience. For variations, try 7z over unzip for robustness. Flag: **flag{s1mple_linux_101}**. Great job navigating the nests!